



Universitat
Oberta
de Catalunya

(HHTMLCSS2) Herramientas HTML y CSS II
PEC 3 - Atomic CSS. Las librerías de utilidades.
Tailwind.

Marc Turu Roca

Profesor: Julián Alarte Aleixandre

Fecha de entrega: 4/6/2026

Universitat Oberta de Catalunya (UOC)

Tabla de contenidos

1. Introducción	2
2. Configuración inicial	2
3. Diseño e implementación	3
3.1 Dependencias	3
3.2 Idea de diseño	4
3.3 Clases extraídas	6
3.4 Componentes extraídos	6
4. Preguntas sobre utility-first CSS	7
5. Análisis de código generado por la IA	8
6. Despliegue	13
6.1 Versionado	13
6.2 Publicación a internet	13
7. Recursos	14
8. Anexos	15

1. Introducción

Esta práctica ha consistido en, partiendo de [UOC Boilerplate](#), crear un sitio web formado por 2/4 páginas desarrolladas en la PEC 2 pero siguiendo el paradigma de **Atomic CSS** usando la librería **Tailwind**, y 1 página a partir de una **IA** generativa.

El repo de **GitHub** es: <https://github.com/marcturu/ac-materia-v2>

El link para acceder a la **web** es <https://ac-materia-v2.netlify.app/>

2. Configuración inicial

En esta primera parte, se hizo un `git clone`

<https://github.com/uoc-advanced-html-css/uoc-boilerplate.git> para poder disponer de la plantilla inicial sobre la cual trabajar y se enlazó al nuevo repositorio creado (<https://github.com/marcturu/ac-materia-v2>) ejecutando:

```
git remote add origin https://github.com/marcturu/ac-materia-v2
```

```
git branch -M main
```

 para cambiar la rama principal.

```
git push -u origin main
```

 para añadir los cambios al repositorio de GitHub.

Seguidamente, se instalaron las siguientes dependencias¹:

```
"@tailwindcss/postcss": "^4.3.0",
"tailwindcss": "^4.3.0"
```

```
"@fortawesome/fontawesome-free": "^7.2.0",
"aos": "^2.3.4"
```

Para la dependencia **Tailwind**, se creó un fichero **.postcssrc**, en el cual se le añadió:

```
{ "plugins": { "@tailwindcss/postcss": {} } }
```

para indicar que cuando se compilara el CSS se debía procesar con Tailwind. Aparte, se instaló la extensión [TailwindCSS Intellisense](#) de Visual Studio Code para añadir algunas funcionalidades que facilitan la edición de código, como el autocompletado, la previsualización de colores o las indicaciones de errores.

Finalmente, para poder usar los iconos de FontAwesome e incorporar el código que generaba Tailwind, se añadió el siguiente fragmento al fichero **_dependencies.scss**²:

```
@import "@fortawesome/fontawesome-free/css/all.css";
@import "tailwindcss/index.css";
```

Debido a que en el enunciado se comenta explícitamente: *“En este proyecto no debes instalar Bootstrap o Stylelint”*, se optó por no realizar ninguna instalación extra.

¹ Las de *Tailwind* usando `--save-dev` para que se procesara en el entorno de desarrollo.

² Para solucionar el error de que no se podía resolver la ruta de FontAwesome, se quitó su `npm:`.

3. Diseño e implementación

3.1 Dependencias

En cuanto a las dependencias externas, primeramente se instaló **FontAwesome** (^2.7.0) para utilizar sus iconos de la siguiente manera:

```
<i class="fa-solid fa-calendar" aria-hidden="true"></i>
```

También se instaló **Tailwind** (^4.3.0), el cual, después de importarlo en las dependencias con `@import "tailwind/index.css"`, se creó un `_tailwind-overrides.scss` donde se personalizaron distintas variables para que Tailwind creara sus clases y se pudieran utilizar en los estilos:

@theme, a partir del cual Tailwind generaba clases básicas (`text-primary`, p.e.):

```
@theme {  
  --color-primary: #bf7233;  
  --color-primary-hover: #c98752;  
  --color-secondary: #2f3e46;  
  --color-secondary-hover: #28353c;  
  --color-accent: #7fa98c;  
  --color-light: #fcfaf8;  
  --color-dark: #1b1b1b;  
  --color-highlight: #e0a458;  
  --color-border: #e6ded3;  
  --font-primary: 'League Gothic', sans-serif;  
  --font-secondary: 'DM Sans', sans-serif;  
  --breakpoint-xs: 480px;  
  --breakpoint-hero: 1100px;  
}
```

Estos cambios permitieron dotar a *Tailwind* de estilos personalizados que seguían un mismo flujo de diseño en torno a la paleta de colores, estilos y layout general, permitiendo regirse a los diseños de la PEC anterior y llevar a cabo la idea de diseño inicial de una manera más fiel.

Por otro lado, se dió uso a las clases propias de *Tailwind*, como por ejemplo `mb-*` para dar margen, `font-bold` para definir el peso del texto, `text-shadow-*` para dotar al texto de sombreado, entre otras.

De esta manera, y aparte de realizar comparaciones visuales, se analizaron los códigos de las páginas realizadas con *Bootstrap* e inspeccionaron las *DevTools* para confirmar qué estilos utilizaba cada elemento para poder transformarlo a *Tailwind*.

Como última dependencia, se instaló la librería externa **AOS** (^2.3.4) con el objetivo de añadir pequeñas animaciones de entrada asociadas al *scroll* y reforzar el carácter visual e inmersivo del sitio web.

La integración de AOS resultó especialmente adecuada porque no requirió de *JavaScript* complejo, funcionó mediante atributos *HTML* declarativos y encajó con la idea de diseño. Es por eso que se decidió incorporar estas animaciones en las tarjetas de los ponentes³ y dotar a sus elementos de dinamismo visual al entrar al *viewport*. Para hacerlo, se añadieron dichas animaciones dinámicamente para aplicarlas de manera distinta dependiendo de la tarjeta del ponente:

```
function applyAOS(isDesktop) {
  getCards().forEach((card, index) => {
    const position = index + 1;

    const animation = isDesktop
      ? position <= 3 ? 'fade-down' : 'fade-up'
      : position % 2 === 1 ? 'fade-right' : 'fade-left';

    card.setAttribute('data-aos', animation);

    if (isDesktop) {
      const delay = index * 100;
      card.setAttribute('data-aos-delay', String(delay));
    } else {
      card.removeAttribute('data-aos-delay');
    }
  });
}
```

3.2 Idea de diseño

Debido a que Tailwind basa sus estilos en la técnica *mobile-first*, se decidió seguir la misma estrategia e ir modificando sus clases declarativas con los *breakpoints* que ofrece: **sm:**, **md:**, **lg:**, **xl:**, **2xl:** (e incluso con uno personalizado **xs:** en 480px).

A pesar de esto, para la parte de la página generada con IA se utilizó un enfoque *desktop-first* debido a que el mockup así estaba representado, por lo que se tuvieron más en cuenta inicialmente los *breakpoints*.

Para las metodologías de estilos, se decidió seguir adoptando [CSS Guidelines](#) como referencia principal, acompañada de las metodologías ITCSS, OCSS y BEM

³ No se añadió a más páginas para no sobrecargar visualmente la interfaz ni perjudicar el rendimiento

(solamente para las clases extraídas con `@apply`). La elección no respondió únicamente a preferencias personales, sino a la necesidad de construir una arquitectura CSS mantenible, escalable y coherente con los requisitos técnicos y metodológicos planteados en esta práctica, aparte de resultar especialmente adecuada al permitir abordar simultáneamente la organización global, la reutilización de estilos y la nomenclatura de componentes.

- **ITCSS** como arquitectura global del proyecto.

Su principal aportación fue organizar los estilos desde las capas más genéricas y de menor especificidad hasta las más concretas y específicas. Este enfoque encajaba especialmente bien con el uso de SASS mediante **parciales** (con los `@use` en las hojas de estilos para importar los parciales y dividir CSS en archivos pequeños), **variables** (con las definiciones `$color-primary: #bf7233` en el archivo de variables), **nesting** (con la anidación de etiquetas y clases CSS siguiendo la jerarquía HTML), `@use` (para importar módulos con *namespace*).

Esta vez no se utilizaron **mixins** debido a que Tailwind ya ofrecía la posibilidad de controlar los estilos según la resolución y sus *breakpoints*.

- **OCSS** para reutilización y separación de responsabilidades.

Su principal aportación fue reutilizar patrones visuales y separar estructura de apariencia. Debido a que en el enunciado se planteaba desarrollar múltiples páginas con *layouts responsive*, OCSS permitió **evitar duplicar** código, mejorar la **consistencia visual** y **reutilizar estructuras** comunes en las páginas sin repetir reglas.

Como ejemplo, fue útil para representar la separación visual de las disposiciones de los contenedores que se podía observar en los *wireframes* y que ya se aplicó en la PEC2.

- **BEM (solamente para las clases extraídas con `@apply`)** para nomenclatura y mantenibilidad.

A pesar de que en la gran mayoría del desarrollo de la aplicación no se utilizó BEM debido a que con *utility-first* se utilizaban clases declarativas y no eran necesarias las declaraciones semánticas de estas, cuando se usó se hizo para **evitar colisiones** entre componentes al mantener una nomenclatura clara, **comprender rápidamente** la función de cada clase al no tener selectores excesivamente anidados y mantener el **código escalable**.

Como ejemplo, fue útil para personalizar los (BE)*Modifiers* de:

`.link-underline`, `&--primary` y `&--secondary` cuando se utilizó el `@apply` para reutilizar estilos en componentes recurrentes.

Debido a que las páginas que se decidieron transformar con estilos Tailwind eran la de los Ponentes y la de Blog, se optó por elaborar un *index.html* en forma de póster. Así, aparte de desarrollar las dos páginas requeridas, se incluía una presentación.

3.3 Clases extraídas

Debido a que en muchas ocasiones había ciertos elementos que compartían un mismo estilo, se decidió crear unas clases comunes para poderlas reutilizar y mejorar el código HTML. De esta manera, se evitaba repetir combinaciones largas de clases en varios sitios y se mantenía cierta coherencia en el diseño. Estas clases extraídas con *@apply* fueron:

1. **.blockquote** para personalizar sus variantes `--primary` y `--accent` en el Blog de manera más individualizada.
2. **.hero-btn** para personalizar sus variantes `--primary` y `--secondary` en los *hero* de la portada y el Blog de cada *background*.
3. **.link-underline** para personalizar sus variantes `--primary` y `--secondary` en el Blog y remarcar la diferencia de *links*.
4. **.card** para utilizar los mismos estilos en las 6 tarjetas de los Ponentes y eliminar código repetido.
5. **.recipe-card** para utilizar los mismos estilos en las 3 tarjetas de la Extra y eliminar código repetido.

Se pueden observar ejemplos de estas extracciones en la sección 4 de los Anexos.

3.4 Componentes extraídos

Al igual que con las clases, hubo partes de código destinadas a una misma función (o incluso reutilizables) que pudieron ser separadas en un único archivo e incluidas gracias a *PostHTML*. Estos componentes fueron:

1. **header.html** y **footer.html** para representar el encabezado y el pie de página de todas las páginas de la web.
2. **footer-nav-contestants.html**, **footer-nav-editions**, **footer-nav-recipes.html** para dividir el *footer* de la última página al tener estructuras de elementos muy similares e individualizables.
3. **home.html** para construir la estructura del *hero* en la portada.
4. **cards.html** para dividir la ordenación de las tarjetas de los ponentes.
5. **hero.html**, **intro.html**, **list.html**, **final.html** para separar las responsabilidades de las secciones que conformaban la página del Blog.

4. Preguntas sobre utility-first CSS

1. ¿Qué diferencias hay entre el enfoque de tipos CSS semántico (el que usaste en las otras PEC) y el CSS de utilidades? ¿Cómo afectó esto a tu proceso de desarrollo? ¿Y a tu código?

La principal diferencia es que en el enfoque semántico se escribe el código respecto a componentes que ofrece Bootstrap en forma de clases, como pueden ser la **Navbar, Button, Toast, Card, Dropdown**,... mientras que en el CSS de utilidades se escribe el código ayudándose de pequeñas clases que sirven de soporte para crear las estructuras de estilos desde la forma más “atómica”, como por ejemplo **mb-*, font-bold, text-shadow-***,...

Los dos enfoques obligan a trabajar de formas diferentes: con CSS semántico, primero pienso en la estructura general y luego aplico los estilos desde los archivos propios. Por otro lado, con el *CSS utility-first* estructuro directamente el diseño en el HTML, por lo que, aunque tenga la ventaja de ser mucho más editable y visible, hace que el archivo quede más cargado de contenido.

En lo que al código respecta, en CSS semántico es más fácil reutilizar estilos globales, y en el CSS de utilidades dependes de la consistencia de clases. En general, se traduce en no tanta cantidad de archivos CSS personalizados y más composición dentro de los HTML.

Además, el uso de largas líneas de CSS de utilidades dificulta la interpretación de los estilos de los elementos y, al inspeccionar las *DevTools*, no aparecen los nombres semánticos que se aplican a cada uno, lo cual dificulta su comprensión.

2. ¿Qué diferencias encontraste entre usar una librería de componentes y una librería de utilidades?

En la librería de componentes se trabaja con componentes ya hechos (como se ha comentado en la pregunta anterior) y listos para configurar, lo cual permite generar estructuras cerradas formadas por elementos ya creados de manera muy rápida.

En la librería de utilidades, se proporcionan pequeñas “piezas” que, combinándolas, sirven para construir una estructura flexible desde 0. Por un lado ofrece más control en la personalización, pero más responsabilidad (al comienzo sobre todo).

3. ¿Qué clases y componentes decidiste extraer y por qué?

Pregunta contestada en la sección 3.3 y 3.4.

5. Análisis de código generado por la IA

Esta fase se trató como la elaboración de una página independiente (como se mostraba en el *mockup* presentado en el enunciado), por lo que se llegó a los resultados desde 0 y con una nueva estructura general diferenciada de la que tenía el resto de la aplicación.

El **primer prompt** enviado fue (**requisitos y estructura**)⁴:

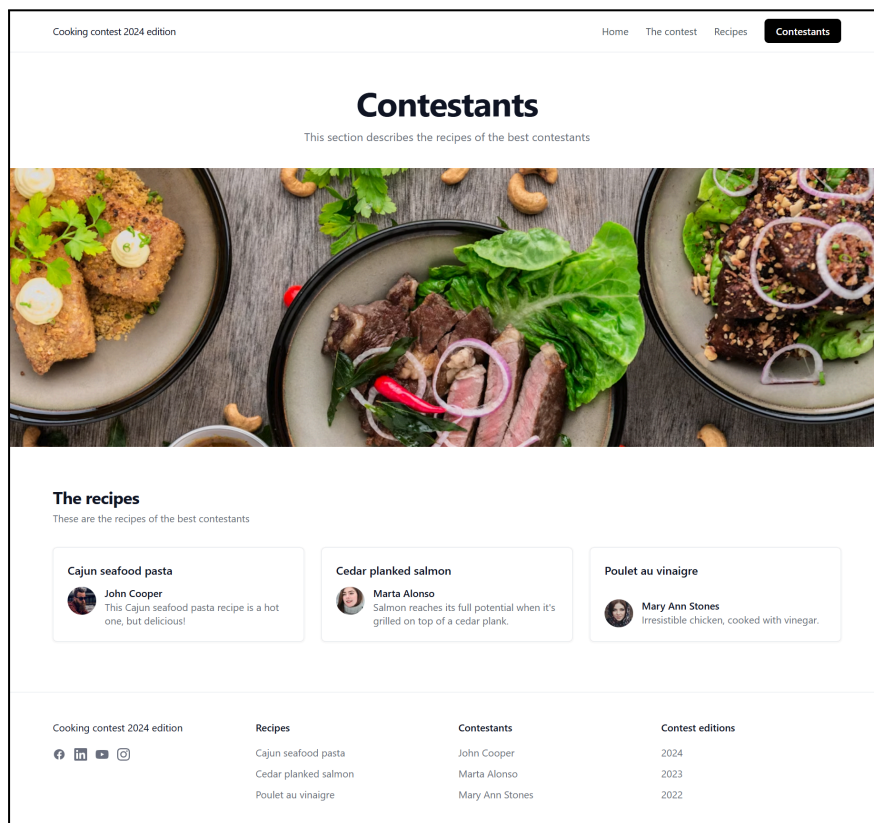
Estoy desarrollando una práctica universitaria utilizando Tailwind CSS v4 dentro de [UOC Boilerplate](#). Necesito convertir un diseño visual en HTML y Tailwind siguiendo el paradigma Utility-First CSS.

Requisitos:

- Utilizar exclusivamente Tailwind CSS.
- No utilizar Bootstrap.
- HTML semántico.
- Accesibilidad básica (*alt*, *aria-label* cuando corresponda, estructura semántica).
- Evitar CSS personalizado salvo casos estrictamente necesarios.
- Priorizar utilidades de Tailwind frente a clases semánticas.
- El resultado debe ser fácilmente integrable en una página HTML.

Voy a adjuntar un PDF con el mockup. Antes de generar código:

1. Analiza la estructura visual.
2. Identifica componentes reutilizables.
3. Propón una estrategia de maquetación.
4. Indica posibles dificultades de implementación.



Presentaba una buena (y sorprendente) estructura general pero no era perfecta ni mucho menos. Las imágenes se resolvieron manualmente con *Google Images*.

Fig. 1 - Resultado de la página Extra después del prompt 1.

⁴ Se pueden comprobar detalles de la respuesta de la IA en la sección 5 de los Anexos.

El **segundo prompt** enviado fue (**ajuste visual y fidelidad al mockup**):

Compara el código generado con el mockup original y realiza una revisión visual exhaustiva. Quiero maximizar la fidelidad respecto al diseño. Analiza especialmente:

- Espaciados verticales y horizontales.
- Márgenes y paddings.
- Anchuras máximas de contenedores.
- Sombras de tarjetas.
- Bordes y radios.
- Tamaños de tipografía.
- Alturas de línea.
- Alineaciones.
- Separaciones entre secciones.
- Diferencias en header y footer.

Corrige todas las discrepancias que detectes. Antes de mostrar el código, enumera los cambios realizados y justifica brevemente cada uno. Mantén exclusivamente Tailwind CSS y evita añadir CSS personalizado salvo que sea estrictamente necesario.

Se pudo observar como el resultado era más fiel al *mockup*, aplicando cambios como por ejemplo:

Eliminado el `border-b border-gray-200` del header, cambiano a `rounded-lg` en el botón activo del header, cambiado a `pt-24 pb-40` en el texto del hero, cambiado a `font-bold` en el h1 del texto del hero, cambiado el estilo inline de la imagen por su correspondencia en *Tailwind* (+ extras para centrar la imagen con un ancho adecuado): `max-w-5xl mx-auto px-6` con `rounded-lg` en la `<img>`, cambiado de `max-w-6xl` a `max-w-5xl` en la sección de las tarjetas para tener el mismo ancho que las otra secciones (aparte de refinar el `pt-20` para separarlo de la imagen), cambiados los tamaños de letra de los dos encabezados de la sección de recetas a `text-3xl` y `text-lg`, respectivamente.

En las tarjetas es donde más cambios hubo (junto con el footer), modificando el gap a `gap-10`, añadiendo el `shadow-lg` (y quitándolo del `:hover`, debido a que la tarjeta no es clickable), añadiendo `items-center` para que la imagen quedase centrada horizontalmente con su texto, y haciendo que ambos últimos textos de la tarjeta fueran `text-xs font-medium`. En el footer, se añadió `max-w-5xl` para limitar el ancho de la sección al mismo que las otras, se redujo ligeramente el espaciado vertical del footer pasando de `p-12` a `py-8`, se aumentó el espacio entre la marca y las redes sociales (en la columna 1) pasando de prácticamente no haber espacio con `gap-6` a designar más con `gap-16`, se remodelaron tamaños (y colores) en la marca: `text-lg`, en las redes sociales (respectivamente): `w-4.5 h-4.5`, `w-3.5 h-3.5`, `w-4 h-4` y `w-3.5 h-3.5` y en los los textos de las tres últimas columnas: `text-xs`. Aparte de dejar estas clases *Tailwind*: `flex flex-col gap-4 text-xs text-gray-600 font-medium` para la lista de ítems de las últimas 3 columnas y reordenar las 4 para que la primera ocupara el 50% (antes ocupaban todas el mismo ancho): `sm:grid-cols-[50%_repeat(3,1fr)]`.

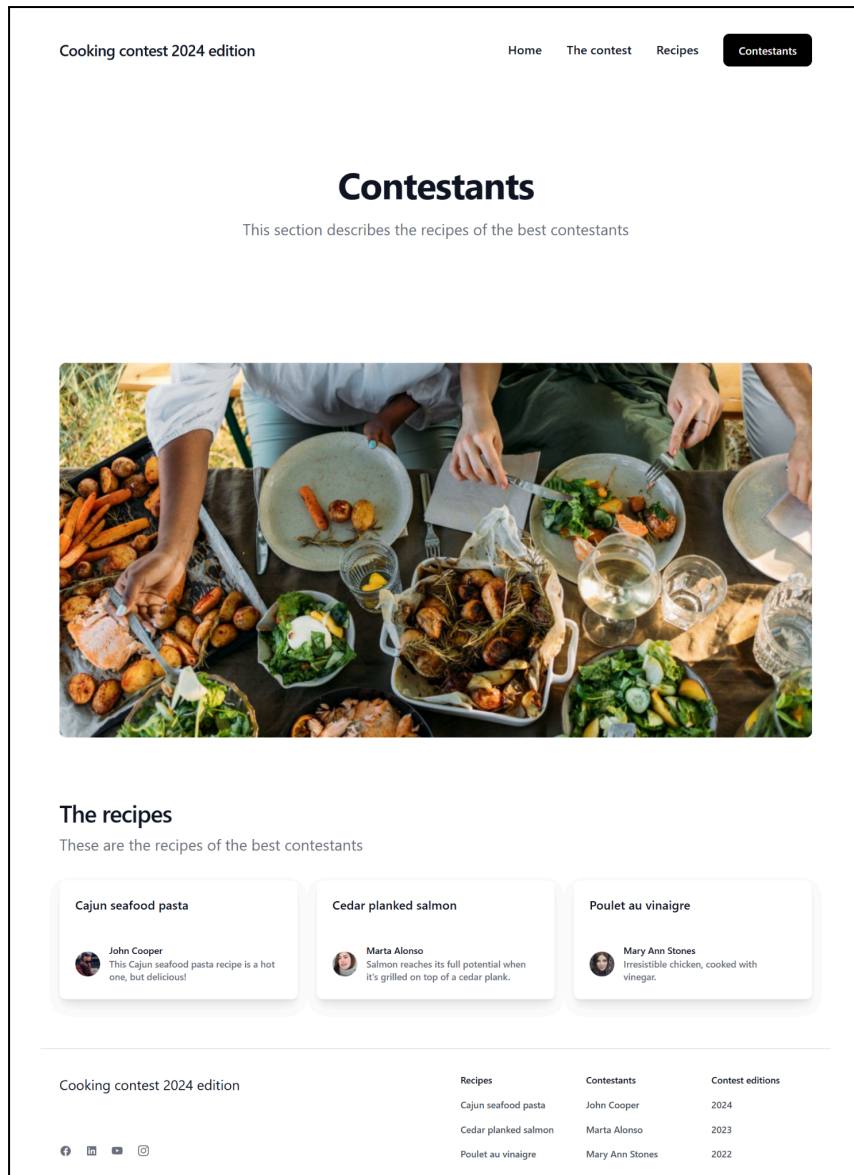


Fig. 2 - Resultado de la página Extra después del prompt 2.

El **tercer prompt** enviado fue (**limpieza, reutilización y adaptación al proyecto**):

Actúa como un desarrollador frontend senior revisando una Pull Request. Analiza el código actual y propón mejoras de mantenibilidad y reutilización. Objetivos:

- Reducir clases repetidas.
- Detectar patrones reutilizables.
- Identificar elementos candidatos a componentes de posthtml-include.
- Identificar clases candidatas a extraerse mediante @apply.
- Simplificar estructuras HTML innecesariamente complejas.
- Mejorar legibilidad del marcado.
- Mantener exactamente el mismo resultado visual. Para cada mejora:
 1. Explica el problema detectado.
 2. Explica la solución propuesta.
 3. Muestra el código resultante. No modifiques el diseño visual ni el comportamiento responsive.

El resultado de este último fue el mismo que el del prompt 2 pero con una mejor organización de código y separación de responsabilidades:

- Las clases `<li>` de las 3 tarjetas repetidas se extrajeron con `@apply` en `_recipe-card.scss` (`.recipe-card`, `&__author`, `&__avatar`, `&__name`, `&__description`) y se utilizaron en el HTML.
- Se crearon `src/partials/extra/footer-nav-recipes.html`, `src/partials/extra/footer-nav-contestants.html` y `src/partials/extra/footer-nav-editions.html` para usarse con `<include src="./views/partials/...html"></include>`, igual que `src/partials/extra/header.html` y `src/partials/extra/footer.html`.
- Para seguir la pipeline de *UOC Boilerplate* pero a la vez no pisarse con los estilos predefinidos para la página principal, se incluyó una hoja de estilos extra `<link rel="stylesheet" type="text/css" href="./assets/styles/extra.scss" />` con: `@use "dependencies"; @use "components/ui/recipe-card";` dentro para usarse específicamente en esa página.

Con una solución muy parecida al *mockup* obtenida con tan solo **3 prompts**, se podían responder las 3 preguntas:

1. ¿Ha cometido la herramienta errores?

Sí, cometió errores sobre todo en aspectos visuales de detalles en el primer prompt. La diferencia entre el primer prompt y el segundo fue que en el primero, a pesar de presentar una estructura general muy sólida, se le escapaban detalles que, si no se le comentaban explícitamente, le costaba corregir.

2. ¿Cuáles son y cuál es la dificultad de corregir el código en contraposición a escribirlo de cero?

Fueron, como se comenta en el segundo prompt, espaciados (tanto verticales como horizontales), márgenes y *padding*s, anchuras máximas de contenedores, sombras de tarjetas, bordes y radios, tamaños de tipografías, alturas de línea, alineaciones y separaciones de secciones, entre otras.

La dificultad fue que, debido a que se parte de un código nuevo no creado por uno mismo o por una persona humana con la que puedes hablar, se tiene que entender y comunicar de otras maneras no tan habituales. Es como refactorizar el código de un compañero pero sin tener la cercanía de poder comunicarte tan fácilmente (aunque la IA posee mucho conocimiento teórico). Primero se tiene que entender la estructura que ha querido crear para después pasar a corregir los errores.

3. ¿Cuál es tu opinión sobre el uso de estas herramientas?

Considero que la IA es una herramienta muy potente y útil para según qué aspectos, sobre todo para realizar tareas repetitivas que pueden llevar algo de tiempo. Aún así, personalmente soy más partidario de utilizarla puntualmente, ya que para la generación de código desde cero (y más aún con Tailwind y sin clases semánticas) no me convence como herramienta única. Necesito sentir que entiendo el código desde la primera línea que escribo, y que no estoy delante de un código sin unas bases sólidas (aunque muchas veces funcional).

6. Despliegue

6.1 Versionado

En el repositorio del código, se mantuvo la rama **main** para los “lanzamientos” de las versiones finales, se creó una rama **develop** para poder juntar las diferentes **features/xx-nombre-feature** y se reservaron las ramas **hotfix/nombre-hotfix** para resolver problemas que surgieran en el código a lo largo del proyecto.

Para publicar los cambios se hacía `$ git add <files> $ git commit -m “<message>” $ git push <branch>`, y para compilar para producción (**/dist**), el comando del `package.json` “build”: “npm-run-all clean parcel:build”: `$ npm run build`.

Branch	Updated	Check status	Behind / Ahead	Pull request
main	2 hours ago		Default	
Your branches				
develop	3 hours ago		4 0	(1 - 21)
hotfix/general	3 hours ago		5 0	
hotfix/theme-apply	3 hours ago		8 0	
feature/85-extra	4 hours ago		10 0	
feature/83-speakers	5 hours ago		11 0	
View more branches >				
Active branches				
develop	3 hours ago		4 0	(1 - 21)
hotfix/general	3 hours ago		5 0	
hotfix/theme-apply	3 hours ago		8 0	
feature/85-extra	4 hours ago		10 0	
feature/83-speakers	5 hours ago		11 0	
View more branches >				

Fig. 3 - Ramas de GitHub

6.2 Publicación a internet

Para poder publicar la web en internet y que estuviera disponible, se tuvo que crear y configurar un nuevo proyecto en **Netlify**. En él, después de especificar el nombre, se estableció la rama **dist/** (la cual era la carpeta que contenía el código optimizado con *Parcel* y listo para producción) como el directorio de publicación y **npm run build** como el comando de construcción. De esta manera, cada vez que se realizaba un cambio en el repositorio en la rama **main**, se creaba un **deploy** con referencia al historial del último **commit** de esta.

7. Recursos

- [Can I use?](#)

Información sobre compatibilidades con navegadores de recursos CSS.

- [Unsplash](#)

Imágenes de portada, ponentes y blog obtenidas de Unsplash bajo licencia gratuita para uso comercial y personal.⁵

- [Squoosh](#)

Compresor y optimizador de imágenes directamente desde el navegador y con variedad de formatos.

- [Canva](#)

Herramienta online de diseño gráfico.

- [AOS — Animate On Scroll](#)

Librería utilizada para las animaciones al hacer scroll.

- [Font Awesome](#)

Librería de iconos utilizada en todo el proyecto.

- [TailwindCSS Intellisense](#)

Extensión de VSCode que añade funcionalidades para facilitar la edición de código.

- [Claude](#)

Asistente de IA generativa creada por Anthropic.

- [Google Images](#)

Funcionalidad de Google para encontrar fotografías, gráficos y otros elementos visuales en internet.

⁵ Se puede observar dicha licencia en los anexos.

8. Annexos

1. **Aplicación web.** Asociación Cultural Materia - Jornadas Artesanales (**Netlify**)

<https://ac-materia-v2.netlify.app/>

2. **Vídeo explicación**

<https://drive.google.com/file/d/1tvKisOr6fl0V1lXo72i6Nk9sQjo7uiRS/view?usp=sharing>

3. **Repositorio con el código (GitHub)**

<https://github.com/marcturu/ac-materia-v2>

4. **Extracción de clase con @apply de .blockquote y .recipe-card:**

```
.blockquote {
  @apply border-l-4 rounded-r-lg px-8 py-4 mb-4 md:-mx-8;

  p {
    @apply italic font-medium !mb-0 text-[clamp(1rem,3cqw,1.35rem)];
  }

  cite {
    @apply text-[0.85rem] font-semibold;
  }

  &--primary {
    @apply border-primary bg-primary/5;

    cite {
      @apply text-primary;
    }
  }

  &--accent {
    @apply border-accent bg-accent/5;

    cite {
      @apply text-accent;
    }
  }
}
```



```
.recipe-card {
  @apply rounded-lg p-5 flex flex-col gap-10 shadow-lg border border-gray-50;

  &__author {
    @apply flex items-center gap-3 mt-auto;
  }

  &__avatar {
    @apply w-8 h-8 rounded-full object-cover shrink-0;
  }

  &__name {
    @apply text-xs font-medium leading-tight;
  }

  &__description {
    @apply text-xs font-medium text-gray-500 mt-0.5 leading-snug;
  }
}
```

5. Respuestas destacadas IA al prompt 1:

1. Estructura visual

El PDF muestra una página de "Contestants" con:

- **Header/Nav** — logo a la izquierda, links de navegación a la derecha, botón activo "Contestants"
- **Hero section** — título grande centrado + subtítulo + imagen de fondo a ancho completo
- **Recipe cards section** — título "The recipes" + subtítulo + grid de 3 tarjetas
- **Footer** — 4 columnas: marca, Recipes, Contestants, Contest editions + iconos sociales

2. Componentes reutilizables

Componente	Reutilización
<nav> con links + CTA	Global en todas las páginas
Recipe card (título + avatar + nombre + descripción)	×3 en el grid
Footer column	×3 columnas de links
Social icon	×4 iconos

3. Estrategia de maquetación

- **Nav:** `flex justify-between items-center` + botón con `bg-black text-white rounded`
- **Hero:** sección con `text-center` + imagen `w-full object-cover` debajo
- **Cards grid:** `grid grid-cols-3 gap-4` con tarjetas `border rounded p-4`
- **Footer:** `grid grid-cols-4` para las columnas

4. Posibles dificultades

- **Tailwind v4:** la configuración y los prefijos pueden diferir de v3 (p.ej. `@import "tailwindcss"` en lugar de directivas `@tailwind`)
- **Avatar circular:** imagen pequeña con `rounded-full` + texto al lado → `flex items-center gap-2`
- **Nav link activo:** el botón "Contestants" tiene estilo `bg-black` diferente al resto de links de texto
- **Imagen hero:** ocupa todo el ancho del contenedor sin afectar al layout de texto superior

6. Licencia Unsplash:

Licencia

Las imágenes de Unsplash se utilizan libremente. Nuestra licencia lo refleja.

- ✓ Todas las imágenes se pueden **descargar** y utilizar de **forma gratuita**
- ✓ **Fines comerciales y no comerciales**
- ✓ **No se necesita permiso** (¡aunque se agradece la atribución!)

Qué no está permitido 🚫

- ⊘ Las imágenes no se pueden **vender** sin modificaciones significativas.
- ⊘ La recopilación de imágenes de Unsplash para replicar un servicio similar o contrapuesto.

Archivos

Unsplash te concede una licencia de derechos de autor irrevocable, no exclusiva y mundial para descargar, copiar, modificar, distribuir, realizar y utilizar imágenes de Unsplash de forma gratuita, incluso con fines comerciales, sin el permiso o la atribución del fotógrafo o Unsplash. Esta licencia no incluye el derecho a compilar imágenes de Unsplash para replicar un servicio similar o contrapuesto.